

리액트 치트시트

1. 리액트 기본

1. React란?

- 👉 컴포넌트와 상태로 UI를 선언적으로 그리는 라이브러리.
- DOM 조작은 직접 안 하고 Virtual DOM이 최소 변경만 반영.
- 데이터 흐름은 단방향이라 추적이 쉽다.
- 재사용 가능한 컴포넌트 조합으로 확장성이 좋다.

2. JSX란?

- 👉 JS 안에서 HTML처럼 쓰는 문법 확장.
- 실제로는 `React.createElement` 호출로 변환된다.
- 하나의 루트만 반환해야 한다.
- `{}` 안에서만 JS 표현식을 사용할 수 있다.

3. 컴포넌트란?

- 👉 독립·재사용 가능한 UI 단위.
- 보통 함수형으로 만들고 `props`를 입력으로 받는다.
- 상태/이벤트를 내부에 캡슐화한다.
- 조합으로 복잡한 화면을 구성한다.

4. props와 state 차이?

- 👉 `props`는 외부 입력, `state`는 내부 변화 데이터.
- `props`는 읽기 전용이고 부모가 제어한다.
- `state`는 `setState`로 컴포넌트 내부에서 바꾼다.
- 데이터 흐름은 항상 부모→자식이다.

5. Virtual DOM이란?

- 👉 메모리상의 DOM 스냅샷.
- 이전/현재 트리를 diff해 변경점만 실제 DOM에 반영.
- 무거운 DOM 조작을 최소화해 성능을 얻는다.
- React 렌더링의 핵심 최적화다.

6. key가 왜 필요한가?

- 👉 리스트 항목 식별을 위한 고유값.
- `key`가 안정적이어야 diff가 올바르게.
- `index key`는 재정렬/삽입 시 버그가 난다.
- 형제 사이에서만 유일하면 된다.

7. Fragment는 언제 쓰나?

- 👉 불필요한 `div` 없이 여러 요소를 감쌀 때.
- DOM에 추가 노드를 만들지 않는다.
- `<></>` 또는 `<Fragment>` 로 표현한다.
- 하나의 루트만 허용되는 JSX 제약을 풀어준다.

8. 조건부 렌더링 방법은?

- 👉 삼항/논리연산자/단락 평가를 쓴다.
- 필요 없으면 `null` 을 반환하면 된다.
- 복잡하면 JSX 밖에서 변수로 분기한다.
- 가독성과 일관성이 중요하다.

9. 이벤트 핸들링 기본?

- 👉 camelCase 속성과 함수 참조를 사용.
- `SyntheticEvent`로 브라우저 간 차이를 숨긴다.
- `event.preventDefault()` 로 기본동작 차단.
- 함수형 컴포넌트라 `this` 바인딩 이슈가 없다.

10. 단방향 데이터 흐름이란?

- 👉 데이터가 위에서 아래로만 흐르는 구조.
- 자식은 props를 직접 수정하지 않는다.
- 변경은 부모 콜백을 통해 상향 전파한다.
- 추적과 디버깅이 쉬워진다.

11. 리렌더링이란?

- 👉 컴포넌트 함수가 다시 호출되는 것.
- Virtual DOM을 만들고 diff해 DOM을 갱신.
- 함수 재실행≠DOM 전체 교체다.
- 변경된 부분만 실제로 그린다.

12. className과 style 사용법?

- 👉 class 대신 className 을 쓴다.
- style은 객체로, 속성은 camelCase다.
- 단위 px는 숫자로 생략 가능.
- 조건부 클래스는 템플릿/도우미로 처리.

13. 컴포넌트 네이밍 규칙?

- 👉 대문자로 시작하는 PascalCase.
- 소문자는 HTML 태그로 인식된다.
- 파일명과 컴포넌트명을 일치시키면 좋다.
- 명확한 역할이 드러나게 이름을 짓는다.

14. SPA란?

- 👉 단일 HTML로 라우팅을 흉내 내는 앱.
- SPA, 스파 (Single Page Application)
- 전환 시 전체 새로고침 없이 일부만 바꾼다.
- 초기 JS 부담이 크지만 UX가 부드럽다.
- Router로 경로별 화면을 관리한다.

15. CSR이란?

- 👉 브라우저가 JS로 렌더링하는 방식.
- 클라이언트 사이드 렌더링 (Client Side Rendering)
- 초기 로딩은 느릴 수 있지만 상호작용이 빠르다.
- SEO는 별도 보완이 필요하다.
- Vite/CRA 기본이 CSR이다.

16. SSR이란?

- 👉 서버가 HTML을 만들어 보내는 방식.
- 서버 사이드 렌더링 (Server Side Rendering)
- 초기 표시와 SEO가 강하다.
- 클라이언트에서 Hydration으로 이벤트를 붙인다.
- Next.js가 대표 구현체다.

17. Hydration(하이드레이션)이란?

- 👉 SSR HTML을 React가 이어받아 활성화하는 과정.
- (CSR은 hydration이 없음. SSR만 있음.)
- 서버/클라이언트 출력이 같아야 한다.
- 불일치 시 hydration 오류가 난다.
- `useId()` 혹은 ID 불일치를 피할 수 있다.

18. 접근성(Accessibility) 기본 원칙?

- 👉 의미 있는 태그와 ARIA 연결.
- label/aria-속성으로 관계를 명확히.
- 키보드 포커스 가능해야 한다.
- 대비/스크린리더 고려가 필요하다.
- 스크린리더(Screen reader)란? 시각장애인이나 저시력자를 위해 컴퓨터나 스마트폰 화면의 텍스트와 정보를 음성으로 읽어주는 소프트웨어

19. 선언형 UI란?

- 👉 무엇을 보여줄지 선언하고 어떻게는 프레임워크가 처리.
- DOM 명령형 조작을 없앤다.
- 상태가 UI를 결정한다.
- 가독성과 유지보수가 좋아진다.

20. 오류 경계(Error Boundary)란?

- 👉 하위 트리 렌더 오류를 잡아 대체 UI 표시.
- 클래스 컴포넌트 기반(함수형은 아직 직접 불가).
- 데이터 에러와는 별개다.
- 사용자 경험을 지켜준다.

2. Hooks

21. 훅이란?

- 👉 함수형 컴포넌트에서 React 기능을 쓰는 함수.
- `use` 로 시작하고 최상단에서만 호출한다.
- 조건/반복 내부 호출은 금지다.
- 호출 순서로 상태가 매핑되기 때문.

22. `useState` 기본?

- 👉 `[state, setState]` 를 반환.
- `setState`는 비동기 배치된다.
- 함수형 업데이트로 최신 값 의존 문제를 해결.
- 초기화는 최초 렌더에만 실행된다.

23. `useEffect` 목적?

- 👉 렌더 이후 부수효과 처리.
- 의존성 배열로 실행 타이밍 제어.
- `cleanup`으로 리소스 정리.
- DOM 접근/구독/타이머/패칭 담당.

24. `cleanup`은 언제?

- 👉 언마운트 직전, 그리고 재실행 전에 호출.
- 이벤트/타이머/구독을 해제한다.
- 메모리 누수와 중복 등록을 막는다.
- 반환 함수에 정리 로직을 둔다.

25. `useLayoutEffect` 용도?

- 👉 DOM 적용 직후, 그리기 전 동기 실행.
- 깜빡임 방지/레이아웃 측정에 사용.
- 과하면 렌더를 막으니 최소화.
- 기본은 `useEffect`로 충분하다.

26. `useRef`는 언제?

- 👉 DOM 접근/변수 캐시/이전 값 보관.
- 변경돼도 리렌더가 일어나지 않는다.
- `state` 대신 임시값 저장소로 사용.
- 포커스/스크롤 제어에 적합.

27. `useMemo`는 언제?

- 👉 무거운 계산 결과 캐싱.
- 의존성 바뀔 때만 재계산.
- 값 캐싱이라 참조도 안정화된다.
- 값이 가벼우면 굳이 쓰지 않는다.

28. `useCallback`은 언제?

- 👉 함수 참조를 캐싱해 자식 리렌더를 줄일 때.
- `React.memo` 와 조합해 효과 극대화.

- 모든 핸들러에 남용하면 역효과.
- 측정 후 필요한 곳에만 쓴다.

29. useReducer 쓰는 때?

- 👉 상태 전이 규칙이 복잡할 때.
- (state, action) => newState 로 일관성 확보.
- 테스트/유지보수가 쉬워진다.
- 작은 규모의 Redux 느낌이다.

30. useContext 주의점?

- 👉 값 변경 시 구독 컴포넌트 전부 리렌더.
- value는 useMemo 로 참조 고정.
- 자주 바뀌는 값은 분리/쪼개기.
- 필요 범위만 Provider로 감싸라.

31. useId는 언제?

- 👉 SSR에서 안정적인 ID가 필요할 때.
- 서버/클라이언트 ID를 일치시킨다.
- 라벨-필드/설명-대상 연결에 유용.
- hydration mismatch를 방지한다.

32. useTransition 개념?

- 👉 낮은 우선순위 업데이트로 표시하는 스케줄링.
- 입력은 즉시, 무거운 갱신은 뒤로 미룬다.
- [isPending, startTransition] 사용.
- 네트워크를 늦추는 게 아니라 렌더 우선순위만 바꾼다.

33. useDeferredValue 개념?

- 👉 값의 지연된 복제본을 제공.
- 읽는 쪽 렌더를 늦춰 입력 반응성 유지.
- 검색 필터/대량 리스트에 효과적.
- useTransition 은 쓰는 쪽, 이걸 읽는 쪽 최적화다.

34. 훅 규칙 두 가지?

- 👉 최상단에서만, 컴포넌트/커스텀 훅 안에서만.
- 순서가 바뀌면 내부 인덱싱이 깨진다.
- ESLint 플러그인으로 강제하자.
- 안정적 상태 매핑의 근간이다.

35. 의존성 배열 다루기?

- 👉 필요한 값만 넣어 재실행 최소화.
- 객체/함수는 참조가 바뀌니 memo/callback으로 고정.
- 누락은 버그, 과다는 불필요 재실행.
- 린트 규칙을 믿고 조정하자.

36. 무한 루프 방지법?

- 👉 effect 안 setState는 의존성 관리 필수.
- 계산 가능한 파생값은 state로 두지 말 것.
- 필요 시 조건문/flag로 실행 제어.
- API 호출은 idempotent 설계.

37. 커스텀 훅 이점?

- 👉 로직 재사용과 관심사 분리.
- 테스트/조합/대체가 쉬워진다.
- UI와 비즈니스 로직을 분리한다.
- 팀 협업에 특히 유리하다.

38. 이전 값 비교 패턴?

- 👉

```
const prev = useRef(value); useEffect(()=>{prev.current=value;}, [value]);
```
- 변화 감지/애니메이션 트리거에 사용.

- ref라 렌더를 유발하지 않는다.
- 간단하지만 강력한 패턴이다.

39. 동기 vs 비동기 effect?

- 👉 useEffect 콜백은 비동기적으로 호출.
- useEffect는 동기 타이밍.
- 비동기 로직은 내부 async 함수로 분리.
- 반환값은 cleanup 함수여야 한다.

40. 상태 초기화 최적화?

- 👉 초기 계산이 무거우면 함수형 초기화.
- useState(()=>expensiveInit()) 형태.
- 최초 렌더에만 계산된다.
- 리렌더 비용을 줄인다.

3. State(상태) & Context

41. 상태는 어디에 둘까?

- 👉 변경 주체와 가장 가까운 공통 부모.
- 너무 높이면 리렌더 전파가 커진다.
- 너무 낮으면 전달이 복잡해진다.
- 최소 범위의 소유자가 정답.

42. 상태 끌어올리기?

- 👉 공통 상태를 부모로 이동.
- 자식끼리 동기화 문제를 없앤다.
- props로 내려 일관성 유지.
- 폼/선택 리스트에서 자주 사용.

43. 전역 상태의 기준?

- 👉 다수 화면에서 공유되고 중요도 높은 값.
- user, theme, locale 등이 예.
- 자주 바뀌면 전파 비용이 크다.
- 꼭 필요한 것만 전역으로.

44. 불변성이 중요한 이유?

- 👉 참조 비교로 변경을 감지하기 때문.
- 원본을 수정하면 리렌더가 안 된다.
- 얇은 비교가 가능한 구조를 만든다.
- spread/immer로 쉽게 유지.

45. 배열/객체 갱신 패턴?

- 👉 map/filter/reduce 로 새 배열 생성.
- 객체는 스프레드로 병합.
- setState는 교체이므로 병합은 직접.
- 중첩은 구조적 업데이트로 분리.

46. 파생값은 state로 둘까?

- 👉 계산 가능하면 state로 두지 말 것.
- 원본에서 즉석 계산이 정합성을 지킨다.
- 필요하면 useMemo로 캐시한다.
- 단일 진실 원천을 유지.

47. Context 기본?

- 👉 props 없이 전역 값 공유.
- Provider→useContext로 소비.
- 테스트용 기본값을 설정해 둔다.
- Provider가 없으면 defaultValue를 반환.

48. Context의 비용과 분리?

- ☞ 값 변경 시 구독자 전부 리렌더.
- Provider를 기능별 분리하자.
- value는 useMemo로 고정.
- setter/상태를 분리 value로 나눠보자.

49. Context에 무엇을 넣나?

- ☞ 자주 안 바뀌는 전역 값 위주.
- 자주 바뀌면 전파 비용이 크다.
- API/캐시 로직은 별도 훅/스토어 권장.
- 최소 데이터만 담자.

50. useReducer + Context 조합?

- ☞ 전역 로직을 명시적으로 관리.
- dispatch로 변경을 통제.
- Provider로 전파해 일관성 유지.
- 복잡도 낮은 전역에는 좋은 선택.

51. selector 패턴 필요성?

- ☞ 필요한 조각만 구독해 리렌더 최소화.
- Context는 기본적으로 전체 재렌더.
- use-context-selector 등으로 보완 가능.
- 외부 스토어가 더 쉬울 때도 있다.

52. Provider 지옥 줄이기?

- ☞ <AppProviders> 컴포넌트로 묶기.
- 기능별 Provider를 합쳐서 감싸기.
- 커스텀 훅으로 접근 추상화.
- 루트의 가독성을 높인다.

53. 로컬 스토리지와 동기화?

- ☞ useEffect로 set/get 수행.
- 초기값은 함수형 초기화로 파싱.
- 변경 시 직렬화해서 저장.
- 키 충돌/버전업도 고려.

54. 비동기 전역 데이터?

- ☞ 로딩/에러/데이터 3상태로 모델링.
- 낙관적 업데이트/재검증 전략을 정한다.
- Suspense 대응 구조를 고민.
- 캐시 계층을 두면 단순해진다.

55. 이벤트 버스 vs 상태?

- ☞ 이벤트는 일회성, 상태는 지속성.
- 전역 상태가 늘어나면 복잡.
- 이벤트로 해결할 문제인지 먼저 판단.
- 기록/디버깅 가능성도 고려.

56. 폼 상태 전략?

- ☞ 컨트롤드 vs 언컨트롤드 선택.
- 라이브러리(react-hook-form) 고려.
- 검증/에러/서브밋 흐름을 분리.
- 성능과 DX(개발자 경험,Developer Experience) 사이의 균형 맞추기.

57. 테이블 대량 편집 상태?

- ☞ 셀 단위 로컬, 선택/정렬은 컨텍스트.
- 가상 스크롤과 memo를 결합.
- 서버 페이징/소팅을 고려.
- 변경 큐로 일괄 커밋.

58. 컴포넌트 간 동기화?

- ☞ 상위 소유로 끌어올려 단일 근원 유지할 것.
- 필요 시 컨텍스트(Context)로 내려준다.
- 두 컴포넌트가 서로 직접 공유하지 않는다.
- 일관성 깨짐(inconsistency)을 방지.

59. 낙관적 업데이트 (optimistic update)

- ☞ (요청이 성공할 거라고 예측하고) 먼저 UI 갱신 후, 만약 실패 시에는 롤백하는(되돌리는) 방식
- 최소 변경 집합을 기록해 되돌린다.
- 사용자 체감 속도가 좋아진다.
- 충돌 해결 전략을 함께 설계해야 함.

60. 상태 명명 규칙?

- ☞ 불변/의도/단위가 드러나게 할 것.
- 로딩/에러/데이터는 접두사로 구분하기
- boolean은 is/has로 시작.
- 도메인 용어를 프로젝트 내에서 일관되게 쓴다.

4. 외부 상태관리 (Redux/Recoil/Zustand/Jotai 등)

61. 외부 스토어가 필요한 순간?

- ☞ 전역 상태가 많아지고 전파 비용이 커질 때.
- Context만으로는 부분 구독이 어렵다.
- 비동기/미들웨어/디버깅이 필요할 수 있다.
- 팀 규모/복잡도에 맞춰 선택.

62. Redux (리덕스) 핵심 흐름?

- ☞ Action→Reducer→Store→View.
- 단일 스토어와 불변성이 원칙.
- 명시적 전이로 예측 가능성이 높다.
- DevTools/미들웨어로 생산성 향상.

63. Redux Toolkit (리덕스 툴킷)

- ☞ Redux의 보일러플레이트 대폭 감소.
- immer/thunk 내장, 표준 구성.
- createSlice/configureStore 로 일관성.
- 팀 협업에 적합한 패턴 제공.

64. Recoil (리코일)

- ☞ atom/selector로 세밀 구독.
- 필요한 컴포넌트만 리렌더.
- 파생값 캐시로 성능 확보.
- React 친화적 API.

65. Zustand(주스탠드)

- ☞ 훅 기반 경량 스토어.
- selector로 부분 구독 간단.
- persist/immer 플러그인 쉬움.
- 소형~중형 앱에 특히 적합.

66. Jotai(조타이)

- ☞ 최소 개념의 atom 스토어.
- 비동기 atom/파생 atom 지원.
- 의존성 그래프가 단순하다.
- 러닝커브가 낮다.

67. RTK Query vs React Query 차이?

👉 전자는 Redux 통합, 후자는 독립 캐시.

→ 둘 다 패칭/캐싱/리트라이 제공.

→ 팀 인프라/선호도에 맞춰 선택.

→ 서버 상태와 클라이언트 상태를 분리.

68. Redux 미들웨어 선택 기준?

👉 Redux Thunk(단순 비동기) vs Redux Saga(복잡한 플로우).

→ Logger/Analytics도 중앙에서 처리.

→ 보안/권한 체크를 일괄 적용 가능.

→ 유지보수 난이도도 고려.

69. selector 성능 전략?

👉 얇은 비교와 메모이제이션 결합.

→ 불필요 구독을 피한다.

→ 중첩 상태는 도메인별로 슬라이스.

→ 계산량이 크면 캐시를 둔다.

70. 마이그레이션 가이드라인?

👉 우선순위 높은 전역부터 외부 스토어로 이전.

→ 로컬 상태는 그대로 유지.

→ API/캐시 레이어를 먼저 도입.

→ 점진적 이전으로 리스크 최소화.

5. 렌더링 & 성능 최적화

71. 렌더링이 느린 이유 찾기?

👉 Profiler로 빈도/시간을 측정.

→ 어떤 컴포넌트가 자주/오래 렌더되는지 본다.

→ 원인 props/state/context를 좁힌다.

→ 데이터/컴포넌트 분할로 해결.

72. React.memo 언제 유효?

👉 같은 props로 자주 렌더될 때.

→ 자식이 무겁거나 리스트 항목일 때.

→ 비교 비용보다 절약 이득이 커야 한다.

→ areEqual로 커스텀 비교 가능.

73. useMemo 남용 금지 이유?

👉 캐시/비교도 비용이 든다.

→ 값이 가벼우면 오히려 느려질 수 있다.

→ 먼저 측정하고 적용하자.

→ 복잡도만 올릴 수 있다.

74. 리스트 최적화 핵심?

👉 안정적 고유 key 사용.

→ 가상 스크롤로 가시 영역만 렌더.

→ 행을 memo하고 셀은 단순화.

→ 서버 페이징/소팅도 고려.

75. diffing과 key 관계?

👉 key로 동일 항목을 매칭해 재사용.

→ 잘못된 key는 상태 꼬임을 부른다.

→ index key는 재정렬에서 취약.

→ 데이터 id가 가장 안전하다.

76. 자동 배칭(React 18)?

👉 여러 setState를 한 번에 커밋.

→ 이벤트/프로미스/타이머에서도 동작.

- 렌더 횟수를 자동으로 줄인다.
- 별도 설정 없이 기본 제공.

77. useTransition 올바른 사용?

- 👉 입력은 즉시, 결과 갱신은 transition으로.
- 무거운 필터/정렬/탭 전환에 적합.
- 블록 안 `setState`만 낮은 우선순위.
- `isPending`으로 진행중 표시.

78. useDeferredValue 케이스?

- 👉 검색어 입력은 즉시 반응.
- 필터링은 지연된 값으로 수행.
- 타이핑 레이턴시를 줄인다.
- UI와 계산을 자연스럽게 분리.

79. Suspense 활용?

- 👉 로딩을 선언적으로 처리.
- `fallback`으로 대기 UI 표시.
- 여러 데이터 요청을 병렬로 묶는다.
- `ErrorBoundary`와 함께 UX 완성.

80. 코드 스플리팅 전략?

- 👉 `route/섹션/컴포넌트` 단위로 분할.
- `React.lazy` + `Suspense` 조합.
- 초기 번들을 가볍게 유지.
- 프리패치로 체감속도 향상.

81. 서버 컴포넌트(RSC) 개념?

- 👉 서버에서 렌더, 클라이언트는 이벤트만.
- 비밀/토큰을 서버에 숨길 수 있다.
- 번들 크기와 네트워크 비용 절감.
- Next.js 13+에서 지원. (RSC를 이해하는 번들러·서버 런타임에서만 동작함)

82. 이미지 최적화 요령?

- 👉 적절한 포맷/사이즈/지연 로딩.
- `<img loading="lazy">` 또는 Next Image.
- `srcset/sizes`로 반응형 제공.
- CDN과 캐시 정책을 병행.

83. 애니메이션 튜닝?

- 👉 `transform/opacity`로 합성 레벨 유지.
- `rAF(requestAnimationFrame)`로 타이밍 동기화.
- 큰 리스트는 `requestIdleCallback` 고려.
- CSS 전환이 JS보다 가볍다.

84. DOM 직접 조작은 언제?

- 👉 React 외부 라이브러리 통합 등 불가피할 때.
- `ref`로 최소 범위만 접근.
- 상태와 DOM의 일관성을 유지.
- 가능하면 선언형으로 대체.

85. 성능 회귀(Performance Regression) 방지 방법은?

- 👉 CI(지속적 통합, Continuous Integration) 시 Lighthouse/Bundle Size 체크.
- 성능 예산을 정해 초과 시 실패.
- 핵심 화면에 E2E(End-to-End) 테스트 측정 추가.
- PR(Pull-Request) 리뷰 체크리스트 운영.

86. 불필요 리렌더 줄이기?

- 👉 props 안정화(`useCallback/useMemo`).
- Context 분리/메모.

- 파생값 즉석 계산/캐시.
- “필요한 곳만” 리렌더되게 구조화.

87. 대용량 테이블 패턴?

- 👉 윈도우잉(Windowing) 기법 + 고정 헤더/열.
- 셀 편집은 로컬, 결과는 배치 커밋.
- 가상화 라이브러리 사용(react-window 등).
- 로딩 중에는 스켈레톤 혹은 프로그레스를 표기

88. 네트워크 튜닝?

- 👉 캐시 정책(stale-while-revalidate).
- 조건부 요청/압축/HTTP2/3.
- 이미지/폰트 프리로드.
- 실패 재시도/백오프 설계.

89. 메모리 누수 방지?

- 👉 cleanup 철저, AbortController로 요청 취소.
- 이벤트/타이머 누락 점검.
- 컴포넌트 unmount 시 자원 해제.
- DevTools Performance로 추적.

90. 최적화의 황금률?

- 👉 측정→원인 특정→최소 개입.
- “보편적 최적화”는 없다.
- 사용자 경로 기준으로 우선순위.
- 유지보수성과 균형을 맞춘다.

6. 라우팅 & 실무

91. React Router 기본?

- 👉 BrowserRouter/Routes/Route/Link 사용.
- URL에 따라 다른 컴포넌트를 렌더.
- 전체 리로드 없이 전환한다.
- 네스티드 라우팅으로 레이아웃 재사용.

92. useNavigate / Navigate 차이?

- 👉 전자는 명령형 이동, 후자는 선언형 리다이렉트.
- 인증/권한에서 조건부 분기 용이.
- replace 옵션으로 히스토리 제어.
- 동작은 History API 기반.

93. Outlet은 언제?

- 👉 부모 레이아웃 안 자식 라우트를 표시할 때.
- 공통 헤더/푸터/사이드바 재사용.
- 중첩 경로를 깔끔히 표현.
- 유지보수가 쉬워진다.

94. 404 처리 방법?

- 👉 path="*" 라우트를 둔다.
- 알맞은 안내/홈 링크 제공.
- 로깅으로 유입 경로 분석.
- SEO 측면의 소프트 404도 고려.

95. 환경 변수 전략?

- 👉 .env.development/.production 분리.
- Vite는 VITE_ 접두사만 노출.
- 민감정보는 서버에서 주입.

- 클라이언트 노출 여부를 항상 점검.
- env 우선순위 : 구체적일수록 높다.

96. 빌드/배포 파이프라인?

- 👉 lint→test→build→deploy 자동화.
- GitHub Actions 혹은 Jenkins로 구성.
- 캐시/병렬로 속도 최적화.
- 실패 알림과 롤백 경로 준비.

97. Bundle/Asset 최적화 방법?

- 👉 Tree Shaking/Code Split/압축.
- 이미지/폰트 서브셋팅(subsetting)하기.
- HTTP 캐시 헤더와 CDN.
- 성능 예산을 수치로 관리.
- 번들(Bundle)이란? : 여러 개의 JS, CSS, 이미지 파일 등의 Asset을 통합해서 최소 개수로 줄인 것.

98. SEO 기본 대응?

- 👉 SSR/메타태그/정적 콘텐츠.
- Open Graph/트위터 카드 설정.
- 크롤링 가능 구조로 빌드.
- Lighthouse/서치콘솔로 점검.

99. React Helmet(리액트 헬멧) 사용

- 👉 페이지별 의미 있는 문서 구조 제공
- <title> 과 <meta> 를 페이지별로 생성 가능
- 스크린리더와 검색엔진이 페이지 맥락을 인식할 수 있음.
- SPA 환경에서도 라우트별 메타데이터 관리 가능.

100. 리액트 코딩 원칙

- 👉 읽기 쉬운 코드가 결국 가장 빠른 코드다
- 상태는 꼭 필요한 곳에만 두고, 전역 상태는 신중히 쓴다.
- 성능 판단은 사용자의 체감 기준(UX)을 우선한다.
- 가장 좋은 코드는 팀 전체가 이해하고 수정할 수 있는 코드다.